

Semana 8: Primeros pasos en Python — De la instrucción al código

Guía completa de la sesión (versión expandida)

Visión general

Duración total	3 horas (1.5 h cátedra con live coding + 1.5 h laboratorio en Colab)
Objetivo central	Que los estudiantes manejen variables, tipos, operaciones, condicionales y bucles básicos en Python, reconociendo que cada concepto viene de la Sección 1
Idea ancla	Python no es un lenguaje nuevo — es la sintaxis para ideas que ya dominan. Hoy cubren el vocabulario básico completo: almacenar datos, tomar decisiones, y repetir acciones
Prerrequisito	Sección 1 completa (especialmente Sem. 1, 2, 4)
Materiales	Proyector, Colab, WiFi, notebook starter compartido vía Notion

ANTES DE LA SESIÓN

- Verificar WiFi (30 conexiones simultáneas).
- Crear y compartir el notebook starter en Notion.
- Aviso previo (Sem. 7): “Necesitan laptop + cuenta Google.”

PARTE 1: CÁTEDRA CON LIVE CODING (90 minutos)

Bloque A — La transición: pseudocódigo → Python (10 min)

NO abrir Python todavía. Proyectar dos columnas:

PSEUDOCÓDIGO (Semana 4)

```
LEER población
SI población < 1000 ENTONCES
    clasificación ← "En peligro"
SINO SI población < 10000 ENTONCES
    clasificación ← "Vulnerable"
SINO
    clasificación ← "Preocupación menor"
FIN SI
ESCRIBIR clasificación
```

PYTHON

```
poblacion = int(input("Población: "))
if poblacion < 1000:
    clasificacion = "En peligro"
elif poblacion < 10000:
    clasificacion = "Vulnerable"
else:
    clasificacion = "Preocupación menor"
# (la indentación cierra el bloque)
print(clasificacion)
```

Pregunta: “¿Cuántas cosas nuevas hay aquí?” — Casi ninguna. = por ←, : por ENTONCES, indentación por FIN SI, elif por SINO SI.

“La lógica es idéntica. Hoy aprendemos las reglas de escritura.”

Bloque B — Colab + primera celda (5 min)

Mostrar rápidamente: notebook nuevo, celda de código, Shift+Enter, comentarios con #.

```
print("Bienvenidos al bosque valdiviano") # primer programa
```

Bloque C — Variables: las celdas de memoria con nombre (15 min)

Creación y asignación (5 min) Callback Sem. 1: “La Memoria del Computador Humano tenía celdas numeradas. En Python, las celdas tienen nombre.”

```
poblacion = 1500
area_km2 = 23.7
especie = "Nothofagus dombeyi"
amenazada = True
print(poblacion)
print(especie)
```

Reglas de nombres: snake_case, no empezar con número, case-sensitive, nombres descriptivos.

Reasignación y operadores de asignación (5 min)

```
conteo = 10
print(conteo) # 10
conteo = conteo + 5
print(conteo) # 15

# Atajos
conteo += 3 # conteo = conteo + 3
print(conteo) # 18
conteo -= 2
print(conteo) # 16
conteo *= 2
print(conteo) # 32
```

“La variable es una celda. `conteo += 3` le dice a la ALU: lee la celda, súmale 3, guarda en la misma celda.”

Asignación múltiple y swap (5 min)

```
latitud, longitud = -39.81, -73.24
print(f"Valdivia: {latitud}, {longitud}")

a, b = 10, 20
a, b = b, a
print(f"a={a}, b={b}") # a=20, b=10

resultado = None
print(resultado) # None
print(type(resultado)) # NoneType
```

Bloque D — Tipos de datos en profundidad (20 min)

Los cuatro tipos básicos (3 min) Callback Sem. 2: “El TIPO le dice a Python cómo interpretar el dato.”

```
edad = 25 # int
temperatura = 15.3 # float
nombre = "Drimys winteri" # str
es_nativa = True # bool
```

Strings en detalle (8 min)

```
especie = "Nothofagus dombeyi"

print(len(especie))          # 19
print(especie[0])            # 'N'
print(especie[-1])           # 'i'
print(especie[0:10])         # 'Nothofagus'
print(especie[11:])          # 'dombeyi'

print(especie.upper())       # 'NOTHOFAGUS DOMBEYI'
print(especie.lower())       # 'nothofagus dombeyi'
print(especie.split())       # ['Nothofagus', 'dombeyi']
print(especie.replace("dombeyi", "pumilio"))
```

“Un string es una secuencia. Cada carácter tiene un índice, empezando en 0 — como las celdas de memoria.”

Conversiones y ASCII (4 min)

```
texto = "42"
numero = int(texto)
print(type(texto), type(numero))
print(numero + 8)          # 50
print("42" + "8")          # "428" - concatenación

# Semana 2
print(ord('N'))            # 78
print(chr(78))             # 'N'
print(bin(78))             # '0b1001110'
```

Booleanos y comparaciones (5 min)

```
print(10 > 5)              # True
print(10 == 5)             # False (== compara, = asigna)
print(10 != 5)             # True
print(10 >= 10)            # True

es_grande = poblacion > 1000
print(es_grande)           # True
print(type(es_grande))     # bool
```

Bloque E — Operaciones: la ALU de Python (10 min)

Aritméticas (3 min)

```
print(10 + 3)              # 13      print(10 // 3)      # 3
print(10 - 3)              # 7        print(10 % 3)       # 1
print(10 * 3)              # 30        print(10 ** 3)     # 1000
print(10 / 3)              # 3.33
```

Comparación (3 min)

```
x = 15
print(x > 10, x < 10, x == 15, x != 15, x >= 10, x <= 10)

= (asignación) vs. == (comparación).
```

Lógicas (4 min)

```
temperatura = 12
lluvia = True
viento_bajo = True
luz_suficiente = True

puede_muestrear = not lluvia and viento_bajo and luz_suficiente
print(f"¿Puedo muestrear? {puede_muestrear}") # False
```

Bloque F — Condicionales: los rombos del diagrama de flujo (15 min)

Callback Sem. 4: “Un rombo en el diagrama era una decisión. En Python, eso es un *if*.”

if/else (3 min)

```
densidad = 27.2
if densidad < 10:
    print(" Densidad baja")
else:
    print(" Densidad normal")
```

if/elif/else (5 min)

```
poblacion = int(input("Población: "))
if poblacion < 250:
    categoria = "En Peligro Crítico"
elif poblacion < 2500:
    categoria = "En Peligro"
elif poblacion < 10000:
    categoria = "Vulnerable"
else:
    categoria = "Preocupación Menor"
print(f"Categoría UICN: {categoria}")
```

Condicionales anidados (3 min)

```
poblacion = 800
tendencia = "decreciente"
if poblacion < 1000:
    if tendencia == "decreciente":
        categoria = "En Peligro Crítico"
    else:
        categoria = "En Peligro"
else:
    categoria = "Vulnerable"
print(f"Categoría: {categoria}")
```

Ejercicio rápido (4 min) “Clasifiquen temperatura: $<5 \rightarrow$ helada, $5-15 \rightarrow$ frío, $>15 \rightarrow$ templado.”

Bloque G — Bucles: las pasadas del Bubble Sort (15 min)

Callback Sem. 4: “Cada pasada del Bubble Sort era un bucle.”

for sobre una lista (4 min)

```
especies = ["coigüe", "canelo", "luma", "ulmo", "lenga"]
for especie in especies:
    print(f"Especie: {especie}")
```

for con range() (4 min)

```
for i in range(5):
    print(f"Parcela {i+1}")

for i in range(1, 6):
    print(f"Cuadrante {i}")

for i in range(0, 100, 10):
    print(f"Transecto a {i} metros")
```

Acumulación: el patrón más importante (4 min)

```
abundancias = [45, 28, 67, 12, 33]

total = 0
for n in abundancias:
    total += n
print(f"Total: {total}")    # 185

# for + if combinados
grandes = 0
for n in abundancias:
    if n > 30:
        grandes += 1
print(f"Especies con >30 ind: {grandes}")    # 3
```

while breve (3 min)

```
intentos = 0
password = ""
while password != "valdiviano":
    password = input("Contraseña: ")
    intentos += 1
print(f"Acceso concedido en {intentos} intentos")
```

“Un while True: sin salida es el Halting Problem de la Semana 5.”

Bloque H — Programa integrador (5 min)

Construir juntos un reporte ecológico que usa TODO: variables, tipos, input, f-strings, for, if/elif/else. Ver el código completo en las diapositivas.

PARTE 2: LABORATORIO EN COLAB (90 minutos)

Nivel 1 — Fundamentos (30 min)

- Ej. 1: Variables y tipos del inventario forestal + cálculos (área basal, H/D)
- Ej. 2: Strings y ASCII — codificar “PUDU” en binario

Nivel 2 — Condicionales (25 min)

- **Ej. 3:** Clasificador UICN completo con if/elif/else + condicionales anidados
- **Ej. 4:** Decisión de muestreo con operadores lógicos

Nivel 3 — Bucles (25 min)

- **Ej. 5:** Abundancias relativas con bucle for (5 especies)
- **Ej. 6:** Filtrar DAPs > 30cm con for + if + acumulación

Desafío (10 min, opcional)

- **Ej. 7:** Shannon H' automático con bucle (N especies, no hard-coded)
-

Notas pedagógicas

Ritmo y priorización

Esta sesión es densa. Si el tiempo se acaba, priorizar: 1. Variables + tipos + operaciones (Bloques C-E) — **imprescindible** 2. Condicionales (Bloque F) — **imprescindible** 3. Bucles for con acumulación (Bloque G) — **muy importante** 4. while + programa integrador — **sacrificable**

La indentación como concepto genuinamente nuevo

La indentación ES la estructura en Python. Mostrar un ejemplo donde la indentación incorrecta cambia el significado. Dejar que Colab auto-indenté después de :.

Errores comunes expandidos

Error	Causa	Solución
NameError	Variable no definida	Ejecutar celdas en orden
TypeError	Operar str con int	Convertir o usar f-string
IndentationError	Sangría inconsistente	4 espacios después de :
SyntaxError	Falta : o = vs ==	Leer el mensaje
ValueError	int("hola")	Validar entrada
IndexError	Índice fuera de rango	Verificar con len()
Bucle infinito	while sin salida	Interrumpir ejecución